Transport Layer

This layer provides end-to-end communication between applications on different devices.

TCP vs UDP

TCP :- Its a reliable, connection-oriented protocol that ensures accurate and ordered data delivery

ex:- missing a small piece of HTML could break the page so HTTP uses TCP

UDP :- Fast, connection less protol and less reliable.

ex:- In an online game, if a player's position update is lost, often better to send a new update immediately rather than wait for the older one to be retransmitted.

TCP detailed

3-way handshake :- Its a process used by TCP to establish a reliable connection between a client & a server before data transfere. It ensures both sides are synchronized and ready to communicate.

SYN, ACK, Seq :- SYN represents "I want to establish a TCP connection". Seq represents the number used for helping synchronization and ACK represents "I received your SYN and i expect your Seq+1 next".

Flow

Client sends SYN=1, Seq=1000 to server

↓

Server receives and sends its own SYN=1, ACK = ~~1000~~ 1001 (client's seq+1) and its own seq which is lets say 2000. Saying i received your SYN and seq and i expect 1001 next and here is my seq

↓

Client receives the SYN-ACK from the Server with the server's seq 2000. Then client sends the final ACK with ACK=2001 (server's seq+1)

↓

Both sides connection established.

Detailed Info

Both client & server uses this seq number for the synchronization.

Example:-

If client's Seq = 1001

data = Hello → 5 bytes

The server can acknowledge

ACK = 1006

1001 = h
1002 = e
1003 = l
1004 = l
1005 = o

order maintained

"I have received bytes upto 1005 and I expect byte 1006 next"

This is how TCP detects missing data and maintains order.

Note:- The word synchronization in this case ~~doesn't represent the general meaning~~, represents "Bring both endpoints into agreement about the Seq numbers & connection state". Basically both client & server have a consistent understanding of the relevant connection information

Also SYN, ACK are considered flags ~~as~~. A flag is just a bit inside the TCP header that is used to signal something. Like 0(off) & 1(on)

So SYN=1 means this segment has the SYN signal turned on.

SYN=0 we just wouldn't represent the same. So even at the last step of handshake along

with the ACK=1, SYN=0 is also sent as there's no need to send SYN=1 the server already knows the client. ~~Just~~ Just know even after the handshake or the at the last step of handshake every flag is always shared. So SYN=0 is always shared.

As soon as the client sends its ACK at the last step of handshake the client reaches the "ESTABLISHED" state which means "I have my own info along with server's needing for this communication". ~~A same~~

~~Those info~~

As soon as the server receives the ACK from client it also reaches this "ESTABLISHED" State.

That info is :-

- Client IP
- Client port
- Server IP
- Server Port
- Seq
- Expected Seq
- ⋮
- etc

Both sides have this info not just the info of ~~the~~ other side

Ports

↳ It is a logical identifier used to distinguish different applications or services on a device, allowing network traffic to reach the correct program.

Basically a way for computer to decide which application should receive incoming network data.

~~Pathway~~ ex:- 443 for HTTPS
80 for HTTP
21 for FTP

[ports are not shared labels they are local identifiers inside each machine]

Temporary ports :- These are used by the client side when initiating connections.

Fixed ports :- Those are used by services that listen for incoming connections.

temporary ports are assigned by OS or chosen by OS but fixed ports are decided in manual configuration & application design.

[Note :- protocols = rules (HTTP, HTTPS)
services = programs using those rules (Nginx, apps, backend systems)]

Example

Youre phone (client)

IP: 192.168.1.10

Instagram server

IP: 157.240.0.35

Using port 443 (HTTPS)
↳ a program that handles incoming requests is actively listening on this port

Flow

↓

1) u open instagram
- youre app needs to fetch youre feed

So it asks OS:
"I need to connect to Instagram server on port 443 (HTTPS)"

2) OS assigns a temporary port:

Client IP: 192.168.1.10
Client port: 53124

youre side = ~~192~~ 192.168.1.10:53124

3) Instagram server is already listening for connections on port 443

4) After the connection is established using TCP youre phone sends data from port 53124 To 443 (of the server)

the server replies from 443 to 53124

Note:- Ur local machine also has fixed ports and can act as a server and use them if you have a service running for incoming connections.

Also the ~~protocol~~ protocols (HTTP, HTTPS, FTP) are decided for port numbers (fixed ports) but ports are mainly used ~~by~~ for applications or programs.

No two applications can use same port at the same time (considering same IP & protocol)

If App A uses 443
App B tries to use 443
OS rejects the second one ("port already in use")

Otherwise the OS wouldn't know which app should receive incoming data for that port

TCP's ordering is not collecting every info then reordering them in the receiving side.

lets say byte 1000 was expected but 1006 arrived first, it waits for other bytes to arrive and holds 1006 in buffers, once the correct bytes arrive it gets handed to programs & later 1006 is handed over. This is how order is mainted.